Deltx: An Intelligent Static Analysis and Predictive Analytics Platform for Software Quality Evolution

Research Proposal

S.M. Silva

Index No: 22001905

Praneesh S.

Index No: 22001591

R.M.I.M. De Silva

Index No: 22001905

Supervisor

Dr. Manjusri Wickramasinghe

University of Colombo School of Computing

Colombo, Sri Lanka

May 27, 2026

List of Acronyms

AI Artificial Intelligence

ARIMA AutoRegressive Integrated Moving Average

BLEU Bilingual Evaluation Understudy

CI/CD Continuous Integration and Continuous Deployment

DivEye AI Detection Framework

EX-CODE Explainable Code Detection Model

GPT Generative Pretrained Transformer

ISO/IEC 25010 International Standard for Software Quality

LIME Local Interpretable Model-agnostic Explanations

LLM Large Language Model

LSTM Long Short-Term Memory

PatchTST Patch Time Series Transformer

SHAP SHapley Additive exPlanations

SonarQube Code Quality Analysis Tool

Squale Software QUALity Enhancement

XAI Explainable Artificial Intelligence

Contents

Chapter 1

Introduction

The modern software engineering landscape is undergoing a paradigm shift driven by the rapid adoption of **Large Language Models (LLMs)** and **AI-assisted programming agents**, which have dramatically increased developer velocity. However, this shift introduces systemic risks: AI-generated code often introduces subtle structural vulnerabilities, architectural inconsistencies, and latent technical debt that evade immediate detection(*Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity* 2025).

Software quality is a multifaceted construct, defined theoretically by international frameworks such as the **ISO/IEC 25010 standard**, which delineates non-functional characteristics including Maintainability, Security, Performance Efficiency, and Reliability (Emergent Mind, 2026). Operationally, these abstract characteristics are measured utilizing static code analysis tools. **SonarQube** is the industry standard for continuous code quality inspection (SonarSource, 2026). Yet, conventional static analysis remains fundamentally retrospective: it evaluates the codebase at a discrete point in time, identifying existing flaws but failing to anticipate how AI-generated code will degrade quality over time.

To address the limitations of retrospective analysis and to accommodate the profound nuances of AI-augmented software development, there is an imperative need for intelligent, predictive analytics platforms. The proposed research project, **Deltx**, aims to architect an expert-level static analysis and predictive analytics

platform built atop the SonarQube engine. By integrating advanced machine learning techniques for stylometric AI-code detection and utilizing state-of-the-art Transformer-based time-series forecasting architectures, Deltx seeks to transform static, isolated metrics into proactive, explainable quality trajectories.

1.1 Problem Statement

Despite the ubiquity of continuous integration and continuous deployment (**CI/CD**) pipelines, the software engineering domain faces a critical research gap in predicting and managing long-term software quality, a challenge specifically exacerbated by the opaque integration of AI-generated code (*On Developers' Self-Declaration of AI-Generated Code: An Analysis of Practices* 2025). This project addresses four interrelated, foundational problems within this domain:

Issue 1: Invisibility of AI-Generated Structural Decay

Current static analysis tools process all source code uniformly, failing to distinguish between human-authored logic and AI-generated code. Recent empirical analyses indicate that AI-generated pull requests contain up to 1.7 times more structural issues than human-authored equivalents, leading to higher issue densities and rapid technical debt accumulation (CodeRabbit, 2026). Without the ability to identify AI-generated code, organizations cannot apply targeted verification, nor can predictive models accurately weigh the decay risks associated with synthetic authorship.

Issue 2: Semantic Gap in Software Quality Metrics

SonarQube generates granular rule-based outputs (cyclomatic complexity, code smells, debt ratios) (SonarSource, 2026). However, there is a distinct semantic gap between these raw, disjointed metrics and high-level business risk frameworks, specifically the ISO/IEC 25010 standard (Emergent Mind, 2026). Translating raw, unweighted metrics into normalized, mathematically sound quality dimen-

sions—such as Correctness, Maintainability, Security, and Efficiency—remains an unsolved architectural challenge. Existing aggregations often fail to handle conflicting signals, meaning a massive volume of trivial formatting warnings can mathematically overshadow a single, catastrophic security vulnerability (*Software quality metrics aggregation in industry* 2026).

Issue 3: Retrospective Nature of Static Analysis

Platforms like SonarQube provide point-in-time snapshots of codebase health but lack time-series forecasting capability, relegating teams to a reactive posture that addresses debt only after it breaches critical thresholds (*Fault Prediction based on Software Metrics and SonarQube Rules. Machine or Deep Learning?* 2021).

Issue 4: Lack of Interpretability in Predictive Metrics

There is a profound lack of interpretability in predictive software metrics. The application of deep learning in software engineering often results in opaque, "black-box" predictions. If a predictive system forecasts a security decline, it must concurrently provide human-readable insights explaining precisely why—identifying the specific historical metrics, code churn events, or AI-code influxes driving the forecast (*A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME* 2023).

By systematically addressing these four challenges, Deltx will advance software engineering theory by combining stylometric AI detection, graph-theoretic bug weighting, advanced time-series forecasting, and Explainable AI into a unified, rigorous framework.

Chapter 2

Literature Review

The theoretical and methodological foundation of Deltx rests upon an extensive synthesis of four distinct but intersecting domains of software engineering and artificial intelligence: **AI-generated text and code detection**, **hierarchical software quality models**, **predictive time-series modeling**, and **Explainable Artificial Intelligence (XAI)**.

2.1 AI-Generated Text and Code Detection

The proliferation of **Large Language Models** has necessitated the urgent development of algorithms capable of distinguishing synthetic text and source code from human authorship. Early approaches (perplexity, token likelihoods, watermarking) have been shown to degrade sharply under domain shifts and adversarial paraphrasing (*Diversity Boosts AI-Generated Text Detection* 2026; *Why AI-Generated Text Detection Fails: Evidence from Explainable AI Beyond Benchmark Accuracy* 2026).

Recent literature emphasizes the necessity of moving beyond surface-level structural indicators toward content-level linguistic, semantic, and surprisal-based features. The **DivEye framework**, for instance, demonstrates that human-authored text exhibits significantly richer variability in lexical and structural unpredictability (surprisal) compared to the highly structured, mathematically optimized out-

puts of LLMs(*Diversity Boosts AI-Generated Text Detection* 2026). Furthermore, foundational research on the **Global Word Distribution** illustrates that AI models, due to their massive, domain-balanced exposure during training (often exceeding trillions of tokens), converge tightly to global probability distributions, resulting in distinct, quantifiable Letter Distribution Signatures(*Beyond Perplexity: Character Distribution Signatures and the MDTA Benchmark for AI Text Detection* 2026).

In the specific context of source code, advanced models like **EX-CODE** leverage language model token probabilities to provide explainable classifications of synthetic code snippets, successfully uncovering the specific syntactic features that differentiate human-written from AI-generated logic(*EX-CODE: A Robust and Explainable Model to Detect AI-Generated Code* 2024).

2.2 Hierarchical Software Quality Models

Concurrently, standardizing the definition and measurement of software quality has been a long-standing objective within the field, culminating in the **ISO/IEC 25010 framework**. This standard defines product quality across eight primary dimensions, including Functional Suitability, Performance Efficiency, Security, and Maintainability, which are further decomposed into specific subcharacteristics(Emergent Mind, 2026). However, bridging the gap between this theoretical framework and the operational metrics generated by static analysis tools like SonarQube is highly complex.

Methodologies such as the **Squale (Software QUALity Enhancement)** model and the **Quamoco project** attempt to resolve this by introducing hierarchical aggregation techniques(*The Quamoco Product Quality Modelling and Assessment Approach* 2012). Squale, in particular, utilizes distinct mathematical formulas to normalize highly divergent metric values into a uniform scale, subsequently applying non-linear weighted averages to aggregate them into higher-level criteria(*Computing the weighted average in Squale (here* $\lambda = 9$*). First, points...* 2026). A critical innovation of the Squale methodology is its explicit penaliza-

tion of severe outliers, preventing a high volume of positive or trivial metrics from masking critical flaws, thus addressing the inherent aggregation paradoxes found in simpler arithmetic models(*An empirical model for continuous and weighted metric aggregation* 2026).

2.3 Predictive Time-Series Modeling

In the domain of predictive modeling for software evolution, research has historically focused on predicting fault-proneness using classical statistical methods, such as **ARIMA**, or traditional machine learning algorithms like **Random Forests** and **Support Vector Machines**(*A software service supporting software quality forecasting* 2026). While **Long Short-Term Memory (LSTM)** networks were later introduced to capture temporal dependencies in code churn and metric evolution, recent comparative literature unequivocally confirms the superiority of **Transformer-based architectures** for long-sequence, multivariate time-series forecasting(*A comparative analysis of LSTM, GRU, and Transformer models for construction cost prediction with multidimensional feature integration* 2026).

Specifically, the **PatchTST (Patch Time Series Transformer)** architecture has revolutionized the field. By segmenting continuous time series into discrete, subseries-level patches, PatchTST retains localized semantic temporal information while quadratically reducing the memory and computational complexity of the attention mechanism(*DecompPatchTST: A Hybrid Framework Fusing Learnable Polynomials and Patch Transformers for Time-Series Forecasting* 2025). Furthermore, its channel-independence design treats each multivariate feature as a separate univariate sequence that shares global weights, proving highly effective in mitigating cross-metric noise in complex, heterogeneous datasets(*PatchTST - Nixtla - Nixtlaverse* 2026).

2.4 Explainable Artificial Intelligence

Finally, the integration of deep learning architectures in risk assessment requires profound transparency to foster developer trust and facilitate actionable remediation. **Explainable AI (XAI)** frameworks, particularly **SHAP (SHapley Additive exPlanations)** and **LIME (Local Interpretable Model-agnostic Explanations)**, have become essential components of modern predictive systems(*Comparing Explainable AI Models: SHAP, LIME, and Their Role in Electric Field Strength Prediction over Urban Areas* 2026).

SHAP utilizes cooperative game theory to calculate the precise marginal contribution of each input feature to the model's final prediction, providing both global and local interpretability with strict mathematical consistency(Testriq, 2026). Applying SHAP to multivariate software defect prediction allows engineers to trace forecasted quality degradations back to their exact historical drivers.

The synthesis of these four domains—AI detection, quality modeling, predictive forecasting, and explainability—provides the comprehensive theoretical scaffolding upon which Deltx is architected.

Chapter 3

Research Objectives / Questions

3.1 Research Objectives and Questions

The primary overarching objective of this research is to conceptualize, design, and theoretically evaluate **Deltx**, a novel architecture integrating static code analysis, AI detection, and predictive modeling. To systematically address the gaps identified in the literature, this project is decomposed into the following specific research objectives:

3.1.1 Research Objectives

1. **AI Code Detection Methodology**

   To define and operationalize an advanced **AI Code Detection** methodology that utilizes structural unpredictability, entropy variance, and stylometric feature distributions (*Beyond Perplexity: Character Distribution Signatures and the MDTA Benchmark for AI Text Detection* 2026) to accurately estimate the proportion of AI-generated code within a repository commit.

2. **Quality Mapping and Weighting Mechanism**

   To formulate a mapping and weighting mechanism that translates granular SonarQube rule violations (SonarSource, 2026) into four primary **ISO/IEC 25010 dimensions** (Emergent Mind, 2026) (Maintainability, Correctness,

Security, Efficiency), explicitly factoring in issue severity, occurrence frequency, code churn (*Use of Relative Code Churn Measures to Predict System Defect Density* 2005), and call-graph centrality (*Centrality in Networks: Finding the Most Important Nodes* 2026).

3. **Conflict-Resistant Quality Scoring System**

   To engineer a conflict-resistant **Quality Scoring System**, inspired by hierarchical frameworks like the Squale model (*The Squale Model - A Practice-based Industrial Quality Model* 2009; *Computing the weighted average in Squale (here $\lambda = 9$). First, points...* 2026), that computes normalized scores on a 0–100 scale (*Min-Max and Z-Score Normalization* 2026; *How to Normalize Data Between 0 and 100* 2026) for each overarching quality dimension, with explicit penalization of outliers (*An empirical model for continuous and weighted metric aggregation* 2026).

4. **PatchTST-Based Predictive Modeling Engine**

   To architect and formalize a **PatchTST-based Predictive Modeling Engine** (*DecompPatchTST: A Hybrid Framework Fusing Learnable Polynomials and Patch Transformers for Time-Series Forecasting* 2025; *PatchTST - Nixtla - Nixtlaverse* 2026) capable of ingesting multivariate historical metric time-series to forecast future quality score trajectories (*A comparative analysis of LSTM, GRU, and Transformer models for construction cost prediction with multidimensional feature integration* 2026), complete with statistically robust confidence intervals (*Fault Prediction based on Software Metrics and SonarQube Rules. Machine or Deep Learning?* 2021).

5. **Interpretation Layer with SHAP Integration**

   To integrate a comprehensive **Interpretation Layer** utilizing SHAP methodologies (*A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME* 2023; *A Comparative Analysis of LIME and SHAP Interpreters With Explainable ML-Based Diabetes Predictions* 2026) to translate numerical quality forecasts into semantic, human-readable insights (Testriq, 2026),

benchmarked against empirically derived industry thresholds.

These objectives are interdependent, collectively transforming static code analysis into a proactive, interpretable quality governance framework.

Chapter 4

Planned Methodology

For our methodology, we are structuring the Deltx platform through a multi-layered approach that moves from basic data ingestion all the way to deep learning forecasting and human-readable interpretation. To keep the scope manageable and consistent, our implementation will focus exclusively on Python repositories.

4.1 System Architecture Overview

Basically, Deltx will act as an intelligent layer sitting directly on top of a standard SonarQube[1] deployment. The architecture is broken down into five main decoupled modules: a feature extraction pipeline, an asynchronous AI code detection module, a quality aggregation engine, a prediction engine, and finally, the interpretation layer.

4.2 AI Code Detection Methodology

Our detection methodology treats AI authorship as a probabilistic signal rooted directly in Large Language Model (LLM) generation mechanics. Because LLMs generate code by continuously maximizing token likelihoods, they leave a distinct, quantifiable statistical signature. We will utilize advanced log-probability-based

[1]SonarQube is a widely using static analysis tool in the software industry

approaches, such as Min-K%[2] Prob and Tag&Tab. By analyzing the log-rank and token likelihood distributions of the code segments, our system can effectively isolate these statistical signatures, allowing us to accurately estimate the percentage of AI-generated code in any given commit without relying on rigid, deterministic rules.

4.3 Bug Categorization and ISO Mapping Model

SonarQube gives us large number of raw, granular rules, but there is a massive semantic gap between those and actual business risk. To be more precise, our objective is instead of indicating x number of warning categorized according to the severity, convey the message to the developer that "security quality dropped by y%" for instance. We are mapping these metrics directly to four core ISO/IEC[3] 25010 dimensions: Maintainability, Correctness, Security, and Efficiency. Rather than just counting bugs, we are weighting every single issue dynamically. These weight factors depends on the issue's severity, its frequency, and most importantly, its call-graph centrality. Which means a vulnerability in a core routing module is heavily penalized compared to an issue in a random testing script.

4.4 Quality Scoring System

This will be our quality index module. Once the issues are mapped and weighted, we normalize them to compute a standardized score from 0 to 100 for each of the four dimensions. A commit may be very good in readability, maintainability but has a very small catastrophic security vulnerability, so the overall score should be heavily penalized based on the severity of that vulnerability. To handle these conflicting situations, we will be using a non-linear aggregation strategy inspired by the **SQUALE**[4] model. This ensures that a single, **critical security** flaw will

[2]Min-K% Prob and Tag&Tab are Pre-training data detection techniques

[3]ISO/IEC 25010 is a software quality model

[4]SQUALE is a software quality assessment model used in tools like SonarQube

drastically drag down the overall system score, preventing it from being masked by thousands of lines of perfectly formatted code.

4.5 Predictive Modeling Engine

The core novelty of our research is shifting from point-in-time retrospective analysis to actual multivariate time-series forecasting. We will be using a state-of-the-art Transformer architecture called **PatchTST**[5] to process historical SonarQube metrics, code churn rates, and our AI-generation percentages. This model will predict the trajectory of the software's quality across future commits, giving engineering teams a proactive heads-up before technical debt gets out of control.

We will not be analyzing code semantics; we will be forecasting the evolution of software health through statistically meaningful, time-aligned quality indicators.

4.6 Interpretation Layer and Explainability

A predictive model is useless if developers do not trust it. To fix the "black-box" problem, we are integrating an Explainable AI (XAI) framework using **SHAP**[6]. If our model predicts a massive drop in maintainability over the next ten commits, this layer will translate that forecast into a human-readable insight, pinpointing exactly which historical metrics or influxes of AI code are driving that specific decay.

4.7 Dataset Design and Strategy

To train our PatchTST predictive engine, we require a massive, temporally dense dataset. Because existing empirical datasets do not track AI authorship alongside continuous code quality metrics, we will construct this specialized dataset from

[5]PatchTST is a transformer based architecture for time-series forecasting

[6]SHAP (SHapley Additive exPlanations) is a powerful framework for interpreting machine learning models

the ground up using a chronological data-mining approach.

We will start by selecting a set of sufficiently large, mature **open-source Python repositories** that possess a deep and extensive commit history. For every single repository, our automated pipeline will sequentially check out the codebase at discrete time intervals; starting right from the repository's initialization.

Upon each checkout, two core processes will run to build a single, comprehensive data row:

- **Authorship Analysis:** We will run our probabilistic AI code detection module across the codebase to calculate the precise percentage of AI-generated code present at that specific timestep.
- **Quality Extraction:** Concurrently, we will execute a SonarQube scan to pull the structural metrics, capturing all existing bugs, vulnerabilities, code smells, and technical debt.

Chapter 5

Evaluation Strategy

To validate the reliability and operational success of the Deltx platform, our evaluation strategy will be focusing on the three main layers of our system.

5.1 AI Code Detection Evaluation

Since our AI detection module relies on likelihood and log-probability-based approaches, we will evaluate its performance against a baseline dataset containing known mixtures of human-written and LLM-generated Python code. We will quantitatively measure its accuracy using standard classification metrics: Precision, Recall, and the F1-Score. Our target is to achieve a baseline that aligns with modern state-of-the-art probabilistic detectors.

5.2 Predictive Modeling Accuracy

The core forecasting capability of our PatchTST engine will be evaluated using standard statistical time-series metrics. We will utilize **Mean Absolute Error (MAE)** to measure the average deviation of our predicted 0-100 quality scores from the actual scores. Additionally, we will calculate the **Root Mean Square Error (RMSE)** to penalize larger, catastrophic forecasting errors—such as predicting a highly stable codebase when a major quality drop can be observed.

5.3 Explainability and Practical Usefulness

Because our predictive model operates strictly within a metric space—analyzing time-series trends of quality scores, AI percentages, and codebase churn—our interpretation layer will focus specifically on temporal and metric feature attribution using SHAP.

Specifically, we evaluate whether the explainability layer can identify the most influential historical time windows and associated metric shifts that contributed to the forecasted decline.

For example, a predicted drop in Maintainability may be attributed to a combination of increased cognitive complexity and elevated AI-generated code proportion occurring within a specific commit window, which the model identifies as having the strongest influence on the forecast.

For the evaluation of this layer, we will be using **human expert validation**, where experienced software engineers will assess the relevance and accuracy of the SHAP explanations in relation to the actual code changes and quality metrics observed in the historical data.

Bibliography

A Comparative Analysis of LIME and SHAP Interpreters With Explainable ML-Based Diabetes Predictions (2026). URL: https://www.diva-portal.org/smash/get/diva2:1886442/FULLTEXT02.pdf (visited on 05/14/2026).

A comparative analysis of LSTM, GRU, and Transformer models for construction cost prediction with multidimensional feature integration (2026). URL: https://www.tandfonline.com/doi/full/10.1080/13467581.2025.2455034 (visited on 05/14/2026).

A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME (2023). arXiv: 2305.02012 [cs.AI]. URL: https://arxiv.org/html/2305.02012v3 (visited on 05/14/2026).

A software service supporting software quality forecasting (2026). URL: https://upcommons.upc.edu/bitstreams/4d1b3444-87ae-4fb2-9434-9963feccf1e3/download (visited on 05/14/2026).

An empirical model for continuous and weighted metric aggregation (2026). URL: https://rmod-files.lille.inria.fr/Team/Texts/Papers/Mord11a-CSMR2011-Squale.pdf (visited on 05/14/2026).

Beyond Perplexity: Character Distribution Signatures and the MDTA Benchmark for AI Text Detection (2026). arXiv: 2605.01647 [cs.CL]. URL: https://arxiv.org/html/2605.01647v1 (visited on 05/14/2026).

Centrality in Networks: Finding the Most Important Nodes (2026). URL: https://webs-deim.urv.cat/~sergio.gomez/papers/Gomez-Centrality_in_networks-Finding_the_most_important_nodes.pdf (visited on 05/14/2026).

CodeRabbit (2026). *AI vs Human Code Gen Report: AI Code Creates 1.7x More Issues*. URL: https://www.coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report (visited on 05/14/2026).

Comparing Explainable AI Models: SHAP, LIME, and Their Role in Electric Field Strength Prediction over Urban Areas (2026). URL: https://www.mdpi.com/2079-9292/14/23/4766 (visited on 05/14/2026).

Computing the weighted average in Squale (here $\lambda = 9$). First, points... (2026). URL: https://www.researchgate.net/figure/Computing-the-weighted-average-in-Squale-here-l9-First-points-05-15-and-3_fig2_258316385 (visited on 05/14/2026).

DecompPatchTST: A Hybrid Framework Fusing Learnable Polynomials and Patch Transformers for Time-Series Forecasting (2025). URL: https://www.mdpi.com/2073-8994/18/3/516 (visited on 05/14/2026).

Diversity Boosts AI-Generated Text Detection (2026). URL: https://openreview.net/forum?id=WscAU9It1l (visited on 05/14/2026).

Emergent Mind (2026). *ISO/IEC 25010 Quality Model*. URL: https://www.emergentmind.com/topics/iso-iec-25010-quality-model (visited on 05/14/2026).

EX-CODE: A Robust and Explainable Model to Detect AI-Generated Code (2024). URL: https://www.mdpi.com/2078-2489/15/12/819 (visited on 05/14/2026).

Fault Prediction based on Software Metrics and SonarQube Rules. Machine or Deep Learning? (2021). arXiv: 2103.11321 [cs.SE]. URL: https://arxiv.org/abs/2103.11321 (visited on 05/14/2026).

How to Normalize Data Between 0 and 100 (2026). URL: https://www.statology.org/normalize-data-between-0-and-100/ (visited on 05/14/2026).

Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity (2025). arXiv: 2508.21634 [cs.SE]. URL: https://arxiv.org/abs/2508.21634 (visited on 05/14/2026).

Min-Max and Z-Score Normalization (2026). URL: https://www.codecademy.com/article/min-max-zscore-normalization (visited on 05/14/2026).

On Developers' Self-Declaration of AI-Generated Code: An Analysis of Practices (2025). arXiv: 2504.16485 [cs.SE]. URL: https://arxiv.org/html/2504.16485v1 (visited on 05/14/2026).

PatchTST - Nixtla - Nixtlaverse (2026). URL: https://nixtlaverse.nixtla.io/neuralforecast/models.patchtst.html (visited on 05/14/2026).

Software quality metrics aggregation in industry (2026). URL: https://cmustrudel.github.io/papers/smr1558.pdf (visited on 05/14/2026).

SonarSource (2026). *Understanding measures and metrics | SonarQube Server - Sonar Documentation*. URL: https://docs.sonarsource.com/sonarqube-server/user-guide/code-metrics/metrics-definition (visited on 05/14/2026).

Testriq (2026). *Explainability Testing in AI: SHAP, LIME & Interpretability Toolkits*. URL: https://www.testriq.com/blog/post/explainability-testing-in-ai (visited on 05/14/2026).

The Quamoco Product Quality Modelling and Assessment Approach (2012). URL: https://teamscale.com/hubfs/26978363/Publications/2012-the-quamoco-product-quality-modelling-and-assessment-approach.pdf (visited on 05/14/2026).

The Squale Model - A Practice-based Industrial Quality Model (2009). URL: https://rmod-files.lille.inria.fr/Team/Texts/Papers/Mord09b-TechReport-Squale.pdf (visited on 05/14/2026).

Use of Relative Code Churn Measures to Predict System Defect Density (2005). URL: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/icse05churn.pdf (visited on 05/14/2026).

Why AI-Generated Text Detection Fails: Evidence from Explainable AI Beyond Benchmark Accuracy (2026). arXiv: 2603.23146 [cs.CL]. URL: https://arxiv.org/html/2603.23146v2 (visited on 05/14/2026).